

RUBY

История и особенности · Запуск программы · Типы и переменные · Функции, рекурсия и замыкания

Команда 5

Тимошенко Дарья
Андреев Ярослав
Гиниятуллина Аида

01 презентация



Аида



Ярослав

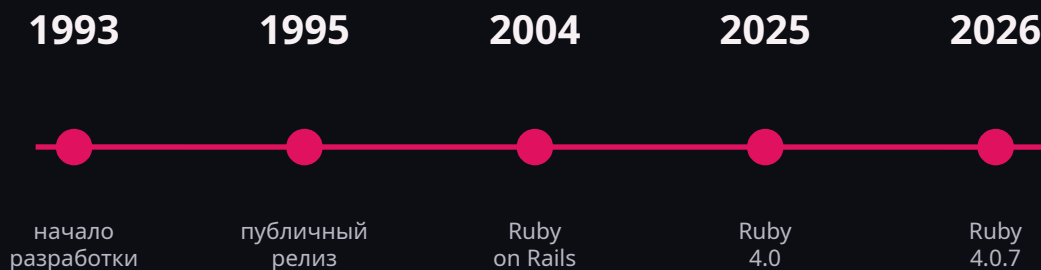


Дарья

История Ruby

От идеи Matsumoto до современной ветки Ruby 4.0

- Создатель - Yukihiro Matsumoto, известный как Matz
- Разработка началась в Японии в 1993 году
- Первая публичная версия появилась в 1995 году
- На Ruby повлияли Perl, Smalltalk, Eiffel, Ada и Lisp
- Главная идея - удобный и естественный язык для программиста



Идея языка

Ruby проектировался так, чтобы код был выразительным и приятным для чтения.

Особенности и спецификация

Ruby сочетает ООП, динамическую типизацию и выразительный синтаксис

- Практически все значения являются объектами
- Типизация динамическая: тип хранится у объекта
- За память отвечает Garbage Collector
- Поддерживаются ООП, императивный и функциональный стиль
- Блоки и итераторы делают код компактным

03_features.rb

```
3.times do  
  puts "Hello!"  
end
```

Ruby 4.0.7 · Open Source

Пример показывает одну из характерных конструкций Ruby: метод `times` выполняет блок кода несколько раз.

Способы запуска - сравнение

Выбор зависит от задачи: демонстрация, разработка или одинаковое окружение

Docker

Плюсы: одинаковая версия Ruby у всей команды, изоляция от системы.
Минусы: нужен установленный Docker.

Онлайн-среда

Плюсы: быстро проверить код без установки.
Минусы: ограничения по файлам, зависимостям и окружению.

Локальное окружение

Плюсы: полный контроль, редактор, RubyGems и Bundler.
Минусы: требуется установка и настройка.

Для примеров используем Ruby 4.0.7

Запуск через Docker

Контейнер помогает получить одинаковую версию Ruby на разных компьютерах

- Готовый Docker Official Image: ruby
- Ruby не нужно устанавливать локально
- У всех участников одна и та же версия среды
- Контейнер можно удалить после запуска

Команды

```
docker run --rm ruby:4.0.7 ruby -e \  
  'puts "Hello, Ruby!'"  
docker build -t ruby-app .
```

Вывод

```
Hello, Ruby!
```

Логика запуска: образ ruby:4.0.7 копирует проект в /app и выполняет main.rb.

Dockerfile

```
FROM ruby:4.0.7  
  
WORKDIR /app  
  
COPY . .  
  
CMD ["ruby", "main.rb"]
```

Онлайн-среда и локальное окружение

Для простых примеров хватит браузера, для проекта удобнее локальная установка

Онлайн

OneCompiler, Replit и другие playgrounds. Подходят для коротких примеров без сложных зависимостей.

Локально

macOS: `brew install ruby`
Linux: `sudo apt install ruby-full`
Windows: RubyInstaller

Проверка и запуск

- `ruby -v` - проверить версию
- `ruby main.rb` - запустить файл
- `irb` - интерактивная консоль

06_hello.rb

```
puts "Hello, Ruby!"
```

Вывод

Hello, Ruby!

Типы данных

В Ruby тип принадлежит объекту, а не переменной

Основные встроенные классы

- Integer - 42; Float - 19.99
- String - "Ruby"
- Symbol - :ruby
- Array - [1, 2, 3]
- Hash - { name: "Matz" }
- nil, true, false - отдельные объекты

07_types.rb

```
number = 42
price = 19.99
name = "Ruby"
language = :ruby

numbers = [1, 2, 3]
person = { name: "Matz", language: "Ruby" }

puts number.class
puts price.class
puts name.class
puts language.class
puts numbers.class
puts person.class
```

Операции над типами

Арифметика, сравнение, логика и работа со строками

Что показывает пример

- Арифметика: +, -, *, /, %, **
- Сравнение: ==, !=, <, >, <=, >=
- Строки можно складывать и интерполировать
- Ложными являются только false и nil
- 0 и пустая строка считаются true

Вывод

```
15  5  8
true true
Hello, Ruby
Ruby 4.0
0 is true
```

08_operations.rb

```
puts 10 + 5
puts 10 / 2
puts 2**3
```

```
puts 10 > 5
puts 10 == 10
```

```
name = "Ruby"
```

```
puts "Hello, " + name
puts "#{name} #{4.0}"
```

```
# В Ruby только false и nil являются ложными значениями.
if 0
  puts "0 is true"
end
```

Приведение типов

Ruby обычно требует явного преобразования несовместимых типов

Основные методы

- `to_i` - преобразование к целому числу
- `to_f` - преобразование к вещественному числу
- `to_s` - преобразование к строке
- `to_a` - преобразование к массиву
- `"10" + 5` даст `TypeError`
- правильно: `"10".to_i + 5`

09_conversion.rb

```
age = "20"

puts age.to_i + 1
puts 42.to_s
puts "3.14".to_f
puts (1..3).to_a.inspect

# "10" + 5 вызвало бы TypeError.
puts "10".to_i + 5
```

Вывод

```
21    42    3.14
[1, 2, 3]
15
```

Области видимости

По имени переменной можно понять, где она используется

Соглашения в именах

- name - локальная переменная
- \$name - глобальная переменная
- @name - переменная объекта
- @@name - переменная класса
- NAME - константа
- новая переменная блока не выходит наружу

10_scope.rb

```
x = 10

1.times do
  x = 20
  y = 30

  puts "Inside: x = #{x}, y = #{y}"
end

puts "Outside: x = #{x}"
```

Вывод

```
Inside: x = 20, y = 30
Outside: x = 20
```

Передача объектов и управление памятью

Модели владения как в Rust в Ruby нет

Главная идея

- переменная хранит ссылку на объект
- при вызове метода копируется ссылка
- изменяемый объект может поменяться внутри метода
- память освобождает Garbage Collector
- не нужно вручную вызывать free/delete

numbers → [1, 2, 3, 4]
array →

11_objects.rb

```
def change(array)
  array << 4
end

numbers = [1, 2, 3]

change(numbers)

p numbers
```

Вывод

[1, 2, 3, 4]

Методы: объявление и вызов

В Ruby результат последнего выражения возвращается автоматически

Что важно

- метод объявляется через `def ... end`
- скобки при вызове часто необязательны
- `return` можно не писать
- последнее выражение становится результатом
- метод `square` возвращает `x * x`

```
Hello, Ruby!  
25
```

Вывод

12_methods.rb

```
def greet(name)  
  "Hello, #{name}!"  
end  
  
def square(x)  
  x * x  
end  
  
puts greet("Ruby")  
puts square(5)
```

Входные данные методов

Ruby поддерживает несколько способов передачи параметров

Варианты аргументов

- обычные позиционные параметры
- значения по умолчанию
- *numbers - переменное число аргументов
- keyword arguments: name:, age:
- удобно, когда у метода много параметров

Вывод

```
Hello, World!  
Hello, Ruby!  
10  
Matz, 61
```

13_arguments.rb

```
def greet(name = "World")  
  "Hello, #{name}!"  
end  
  
def sum(*numbers)  
  numbers.sum  
end  
  
def user(name:, age:)  
  "#{name}, #{age}"  
end  
  
puts greet  
puts greet("Ruby")  
  
puts sum(1, 2, 3, 4)  
  
puts user(name: "Matz", age: 61)
```

Выходные данные методов

Ruby умеет автоматически возвращать последнее выражение

Что важно

- return можно не писать
- возвращается результат последнего выражения
- несколько значений удобно вернуть массивом
- результат можно распаковать в переменные

Вывод

```
Min: 1  
Max: 5  
Square: 25
```

14_return_values.rb

```
def minmax(numbers)  
  [numbers.min, numbers.max]  
end  
  
def square(x)  
  x * x  
end  
  
min, max = minmax([3, 1, 4, 1, 5])  
  
puts "Min: #{min}"  
puts "Max: #{max}"  
puts "Square: #{square(5)}"
```

Рекурсия

Метод может вызывать сам себя, но обязательно нужно условие остановки

Главная идея

- метод factorial вызывает сам себя
- условие $n \leq 1$ останавливает рекурсию
- factorial(5) возвращает 120
- Ruby по умолчанию не гарантирует оптимизацию хвостовой рекурсии
- для глубокой рекурсии безопаснее цикл или итератор

15_recursion.rb

```
def factorial(n)
  return 1 if n <= 1

  n * factorial(n - 1)
end

puts factorial(5)
```

120

Вывод

Блоки и итераторы

Блок - фрагмент кода, который можно передать методу

Что видно в примере

- `do ... end` задаёт многострочный блок
- между `|` `|` записываются параметры блока
- `times` выполняет блок несколько раз
- `each` по очереди передаёт элементы массива
- блоки делают работу с коллекциями компактной

16_blocks.rb

```
3.times do |i|  
  puts "Iteration #{i}"  
end
```

```
numbers = [1, 2, 3]
```

```
numbers.each do |number|  
  puts number * 2  
end
```

Вывод

```
Iteration 0  Iteration 1  Iteration 2  
2    4    6
```

Замыкания: Proc и lambda

Замыкание сохраняет доступ к переменным внешней области

Что важно

- Proc и lambda хранят блок кода как объект
- вызываются через .call
- counter возвращает lambda
- переменная count сохраняется между вызовами
- так можно хранить состояние без отдельного класса

Вывод

```
1  
2  
3  
10
```

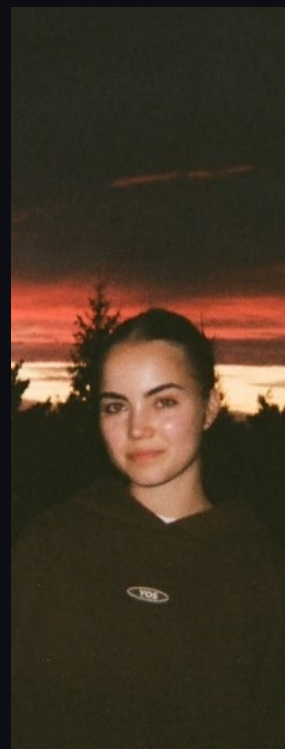
17_closures.rb

```
def counter  
  count = 0  
  
  -> {  
    count += 1  
  }  
end  
  
counter1 = counter  
  
puts counter1.call  
puts counter1.call  
puts counter1.call  
  
double = Proc.new { |x| x * 2 }  
  
puts double.call(5)
```

Спасибо за внимание!

Мы рассмотрели историю и особенности Ruby, способы запуска программ, типы данных, методы, рекурсию, блоки и замыкания.

Вопросы?



Аида



Ярослав



Дарья

Команда 5 · Тимошенко Дарья · Андреев Ярослав · Гиниятуллина Аида